

Sentiment Classification for Social Media

CS918 - Assignment Two

U2104970: Mohammed Abdullah

March 2025

1 Introduction

Sentiment analysis is an integral task in natural language processing (NLP), involving the computational assessment of textual content to determine the expressed sentiment. In this project, I developed a classification system for sentiment analysis on Twitter data, specifically targeting the SemEval 2017 Task 4 Subtask A dataset¹. The objective was to build a robust classifier capable of accurately categorizing tweets as positive, negative or neutral.

2 Journey Through the Project

2.1 Data Preprocessing and Feature Engineering

I began with extensive preprocessing steps, drawing on ideas from a previous assignment. These steps included:

- Retaining only alphanumeric characters and spaces.
- Removing words with only one character, as such tokens (typically stopwords) rarely contribute meaning.
- Eliminating purely numeric tokens to avoid confusion in a bag-of-words (BoW) framework.
- Removing URLs that do not contribute semantic value and could disrupt tokenization.

2.2 Classical Classifier Development

2.2.1 SVM with Bag-of-Words Features

I initially implemented a support vector machine (SVM) classifier using TF-IDF weighted BoW features. Early experiments involved randomized grid search for hyperparameter tuning, but issues with infeasible parameter combinations necessitated a switch to a standard grid search with a refined parameter grid.

¹<https://alt.qcri.org/semeval2017/task4/index.php?id=data-and-tools>

2.2.2 MaxEnt Classifier with BoW Features

Subsequently, I developed a Maximum Entropy (MaxEnt) classifier using Logistic Regression with the same BoW features. Minor convergence issues were addressed by increasing the maximum iteration count. After realizing that normalizing the feature vectors was essential to ensure that all features contributed equally, I incorporated normalization, which significantly improved performance.

2.3 Word-Embedding Based Features

2.3.1 GloVe Embeddings

To capture richer semantic representations, I explored using GloVe embeddings. I experimented with average GloVe features for both SVM and MaxEnt classifiers. Initial attempts with plain averaging performed worse than BoW features. I then explored weighted averaging, but it did not yield improved performance. Ultimately, I combined average pooling with max pooling to retain both overall semantic context and extreme features. This approach improved performance notably. Additionally, setting the class weight to “balanced” in the MaxEnt classifier further enhanced the model’s ability to handle class imbalances.

2.4 LSTM-based Classifier

I then implemented a Long Short-Term Memory (LSTM) network. Using PyTorch and custom data loaders, I constructed an LSTM architecture as specified. Hyperparameter tuning was performed with careful monitoring of training and generalization error on a development set. The optimal number of epochs was determined by identifying the minimum in the validation loss curve. Interestingly, the LSTM classifier only marginally outperformed the BoW-based SVM, possibly due to the inherently limited context of tweets.

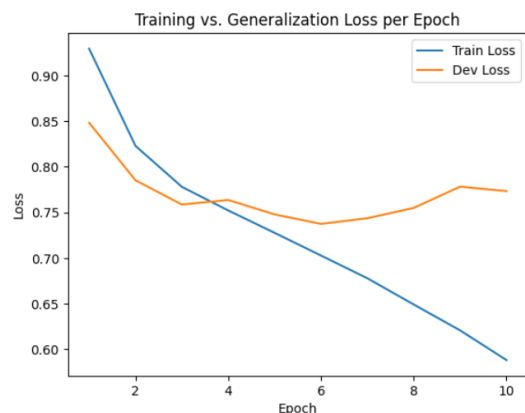


Figure 1: Plot of training and generalisation losses against epochs, notice the minimum in generalisation error around epoch 6

```
Starting LSTM training...
Epoch 1/10 - Train Loss: 0.9294, Dev Loss: 0.8482, Time: 29.26s
Epoch 2/10 - Train Loss: 0.8228, Dev Loss: 0.7851, Time: 36.81s
Epoch 3/10 - Train Loss: 0.7780, Dev Loss: 0.7586, Time: 37.58s
Epoch 4/10 - Train Loss: 0.7521, Dev Loss: 0.7636, Time: 37.94s
Epoch 5/10 - Train Loss: 0.7276, Dev Loss: 0.7478, Time: 39.06s
Epoch 6/10 - Train Loss: 0.7027, Dev Loss: 0.7374, Time: 39.15s
Epoch 7/10 - Train Loss: 0.6778, Dev Loss: 0.7436, Time: 39.53s
Epoch 8/10 - Train Loss: 0.6491, Dev Loss: 0.7549, Time: 38.23s
Epoch 9/10 - Train Loss: 0.6207, Dev Loss: 0.7782, Time: 37.93s
Epoch 10/10 - Train Loss: 0.5882, Dev Loss: 0.7733, Time: 37.79s
```

Figure 2: Data for 3

2.5 Transformer-Based Classifier: BERT

Recognizing the limitations of classical and LSTM approaches, I developed a classifier based on BERT. BERT is a transformer-based model that employs bidirectional context to generate deep linguistic representations. I fine-tuned BERT using GPU resources on Google Colab and later exported the model for local inference. Initial fine-tuning resulted in BERT predicting only “neutral” due to a formatting issue in the data; after refining the data loading function, this problem was resolved. However, inference on a local machine proved slow, leading to additional fine-tuning with more layers unlocked and subsequent dynamic quantization. Quantization, which reduces numerical precision and model size, significantly sped up inference with little loss in accuracy. The final quantized BERT classifier achieved a macro-F1 score of approximately 0.713, outperforming all previous models and even exceeding the best reported score in the original task.

```

Training completed in 843.239 seconds.
Saving full model to /content/drive/MyDrive/bert_model_full
Full model saved.
Performing dynamic quantization on the model...
Tracing quantized model...
Saving quantized model to /content/drive/MyDrive/bert_model_quant
Quantized model saved.
Total training time: 843.428 seconds.
adding: content/drive/MyDrive/bert_model_full/ (stored 0%)
adding: content/drive/MyDrive/bert_model_full/config.json (deflated 51%)
adding: content/drive/MyDrive/bert_model_full/model.safetensors (deflated 7%)
adding: content/drive/MyDrive/bert_model_full/tokenizer_config.json (deflated 25%)
adding: content/drive/MyDrive/bert_model_full/special_tokens_map.json (deflated 42%)
adding: content/drive/MyDrive/bert_model_full/vocab.txt (deflated 53%)
adding: content/drive/MyDrive/bert_model_quant/ (stored 0%)
adding: content/drive/MyDrive/bert_model_quant/bert_model_quant.pt (deflated 23%)
adding: content/drive/MyDrive/bert_model_quant/tokenizer_config.json (deflated 25%)
adding: content/drive/MyDrive/bert_model_quant/special_tokens_map.json (deflated 42%)
adding: content/drive/MyDrive/bert_model_quant/vocab.txt (deflated 53%)
Full model zipped as bert_model_full.zip and quantized model zipped as bert_model_quant.zip. Download these files for later use on your laptop.

```

Figure 3: Model quantization and exporting

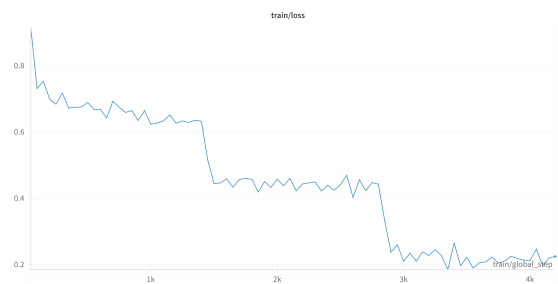


Figure 4: Detailed plot of training loss per step

2.6 Few-Shot Prompting with Language Models

Inspired by recent advancements, I experimented with few-shot prompting strategies using GPT-2 and later Flan-T5. I created stratified test subsets to allow for rapid evaluation. I ensured that the few-shot prompt included examples from all three sentiment classes. While GPT-2 initially produced unsatisfactory results, switching to an instruction-tuned model such as Flan-T5 resulted in significant performance improvements, emphasizing the impact of prompt design and model selection.

3 Results

Table 1 summarizes the performance of the various classifiers on the SemEval 2017 test sets. In addition to the individual scores for Test 1, Test 2, and Test 3, I computed the mean and standard deviation across these test sets for each classifier.

Classifier	Test 1	Test 2	Test 3	Mean	Std
BoW + SVM	0.601	0.608	0.567	0.592	0.018
Avg-GloVe + SVM	0.418	0.414	0.449	0.427	0.016
BoW + MaxEnt	0.538	0.538	0.519	0.532	0.009
Avg-GloVe + MaxEnt	0.562	0.575	0.533	0.557	0.018
LSTM	0.586	0.592	0.546	0.575	0.020
Quantized BERT	0.721	0.714	0.704	0.713	0.007
Prompting (few-shot)*	0.750	0.727	0.656	0.711	0.040

Table 1: Performance of sentiment classifiers. The last two columns present the mean and standard deviation across the three test sets. * Scores for the prompting classifier are based on a stratified subset of the test data, which may impact the overall performance.

4 Discussion of Results

Table 1 summarizes the comparative performance of sentiment classifiers evaluated on the SemEval test datasets. Classical methods using bag-of-words (BoW), such as SVM and MaxEnt, provided stable baseline performance. However, the relatively modest scores (mean F1 scores of 0.592 for SVM and 0.532 for MaxEnt) highlight inherent limitations in representing nuanced sentiments through shallow lexical features alone.

Experimentation with GloVe embeddings revealed that simplistic averaging of embeddings does not adequately capture contextual meaning, as evidenced by lower scores (mean scores between 0.418 and 0.557). Incorporating max pooling alongside average embeddings yielded modest gains by preserving more distinct semantic cues; however, this hybrid approach still underperformed relative to classical BoW methods, emphasizing challenges in effectively integrating word embeddings.

The modest improvement of the LSTM-based classifier (mean F1 of 0.575) compared to classical methods highlights that, despite theoretical advantages, recurrent architectures may struggle with the limited contextual information inherent in brief tweet texts. This suggests that sequence-modeling benefits of LSTMs are less impactful in shorter textual data.

Notably, the quantized BERT classifier significantly surpassed all prior methods, achieving a mean macro-F1 score of 0.713. This confirms the superior capability of transformer architectures in capturing deep, contextual relationships. The applied quantization optimized inference efficiency without compromising accuracy, reinforcing its practical relevance.

The few-shot prompting approach demonstrated competitive mean performance (0.711), yet the higher variability (standard deviation 0.040) highlights its sensitivity to prompt quality and the reduced evaluation sample sizes. Despite these limitations, prompting-based strategies present promising avenues for future exploration, particularly with instruction-tuned models.

Overall, these results emphasize the clear progression in NLP methodologies, demonstrating the considerable benefits of employing advanced transformer-based models and innovative prompting techniques over classical and simpler embedding-based approaches.

5 Conclusion

This project systematically explored various methodologies for Twitter sentiment analysis, transitioning from classical machine learning techniques to advanced neural network models and innovative prompting approaches. Key insights from the development process include:

- Effective preprocessing techniques, particularly noise reduction, are foundational for accurate classification.
- Feature normalization significantly enhances the performance of classical models.
- Complex sequential models such as LSTMs provide limited benefit for brief texts due to constrained contextual information.
- Transformer-based models, specifically fine-tuned and quantized BERT, substantially outperform simpler methods by leveraging deeper contextual representations and optimizing inference speed.
- Few-shot prompting techniques, when carefully designed, offer promising yet variable performance, influenced heavily by prompt construction and dataset characteristics.

The final quantized BERT classifier achieved an impressive mean macro-F1 score of 0.713, surpassing the top original SemEval 2017 submission (0.6851) [1]. These findings underscore the effectiveness and practical advantages of transformer-based approaches in NLP and sentiment analysis. Future research should focus on further model distillation, enhanced few-shot and zero-shot prompting strategies, and hybrid ensemble approaches to further exploit recent advances in transformer architectures and improve sentiment classification accuracy.

References

- [1] Semeval-2017 task 4: Sentiment analysis in twitter, 2017. <https://alt.qcri.org/semeval2017/task4/data/uploads/semeval2017-task4.pdf>.